

Optimizing Pattern-Matching Compilation Over OCaml's Extensible Variant Types

Maël Coulmance, Vincent Laviron, Pierre Chambart

▶ **To cite this version:**

Maël Coulmance, Vincent Laviron, Pierre Chambart. Optimizing Pattern-Matching Compilation Over OCaml's Extensible Variant Types. JFLA 2026 – 37es Journées Francophones des Langages Applicatifs, Marie Kerjean; Yannick Zakowski, Jan 2026, Oberbronn, Alsace, France. ⟨hal-05428149⟩

HAL Id: hal-05428149

https://hal.science/hal-05428149v1

Submitted on 21 Dec 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Copyright - All rights reserved

Optimizing Pattern-Matching Compilation Over `OCaml`'s Extensible Variant Types

Maël Coulmance[1,2], Vincent Laviron[1], and Pierre Chambart[1]

[1]OCamlPro, Paris, 75014, France
[2]Université Paris Cité, Paris, 75013, France

Extensible variant types have been part of *OCaml* since version *4.02* and still, as of version *5.4* have a naive pattern matching compilation schema. This paper explores different approaches to compile, potentially more efficiently, those pattern matchings. We propose three compilation schemes, two of which have been implemented in a fork of the compiler, and we discuss the benefits of such optimization for the future of *OCaml*.

1 Introduction

The idea of having extensible data types is not new [Ler90]. In *OCaml* itself, the type of exceptions has been extensible from the start, allowing users to declare new exceptions, raise them, and match on them, with all the type safety one would expect from a functional language.

There are traces in the *OCaml* folkore of uses of the exception type for the sole purpose of adding new constructors to an existing type. This came with obvious drawbacks: the exception type already comes with a number of constructors, and the additional indirection may be considered unpleasant.

In version `4.02`, *OCaml* added support for generic extensible types, allowing users to define their own types in the most natural way. This reused the compilation scheme used for exceptions, with little changes outside the typing and parsing code. This scheme linearly checks the equality of the scrutinee with each case in a pattern. While this might be well-suited for pattern-matching on a small set of constructors, it also might feel disapointing compared to the compilation scheme for classical variants in *OCaml* that produces either a single jump and load or a binary decision tree.

We think that users might refrain from using extensible types when this would be the best solution, due to some potential higher runtime costs. For instance it is often mentioned in *OCaml* folklore that polymorphic variant are more expensive than regular ones. While this is true, the difference is far smaller than between regular and extensible. This limits the use cases for extensible types; for instance, encoding complex protocols designed for extensibility, or representing primitive operations in a complex, low-level language that regularly grows, are problems that would be elegantly encoded using extensible types but the performance overhead will likely convince programmers to look for another solution.

This paper investigates possible alternative compilation schemes for extensible types, in the hope that they become the natural way to encode extensible data structures no matter their complexity.

```
type t = ..
type t += A | B of int * int
type t += C=B

let x = A
let y = C (42, 7)
```

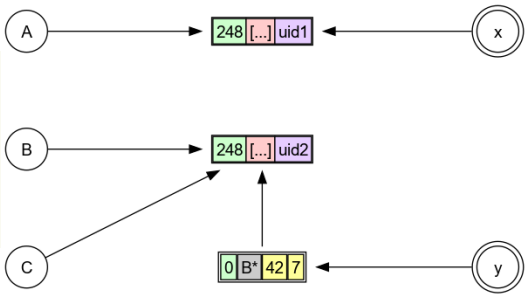


Figure 1. Memory layout for a basic example

1.1 Definition

An *extensible variant type* (or *open type* for short) is a sum type for which the set of its constructors can be extended during the execution of the program. We call an *extension* the *OCaml* declaration that extends an open type.

The exact set of extensions is not something that can be statically known and can depend on the program input. We want to make the reader aware that a whole program analysis wouldn't be sufficient to know the exact set of declarations. For instance a function could declare a first class module containing extensions, and it would be impossible to know how many times that function is called.

For historical reasons, a type extension can also bind a new constructor to a previously defined one.[1] This can be used either to expose the same feature under different names, or to re-export in the current scope an extension defined elsewhere. The new extension then becomes indistinguishable from the previous one. In particular, matches on either of the names will catch values created with the other name.

1.2 Memory Layout

Type extensions, unlike regular type definitions and polymorphic variants, have a runtime component. More precisely, each declared constructor will have a runtime value associated to it.

- "Fresh" constructor declarations are compiled into a block allocation. This block will have tag 248 and will store (in addition to other stuff that are irrelevant here) a *unique* identifier. This identifier will be created at runtime, and we have no way of knowing its value during compilation.
- *Rebinded* constructor declarations are compiled into an alias to the block of the rebindee.

When compiling value creation we also fall into two cases:

- *Constant Constructors* (i.e. constructors that **do not** have a payload): Values are compiled into an alias to the block of the related constructor
- *Non-Constant Constructors* (i.e. constructors that **do** have a payload): Values are compiled into block allocation. This block will have tag 0 and will store in its first field a reference to the block of the related constructor, and its payload in the other fields.

[1] However, there seems to be almost no use of this feature in the wild, except in the OCaml standard library for predefined exceptions.

1.3 Pattern-Matching

Compilation of pattern-matching is divided into three main steps. First, we break down complex patterns into simpler ones, allowing us to rewrite the pattern matching into a combination of high-level switches (conditional branches based on a single value)[FM01], [Mar08]. Then we transform each high-level switch into a bunch of conditional statements. Finally we transform the resulting sequences into either bytecode or native code.

The part that we are interested in is the second one. Indeed, the first step is not particularly related to extensible variants (it is common for all pattern-matching) and is already well optimized, while the third one is specific to a given backend meaning that we would need to duplicate the work for each implementation.

For extensible variants, the second step must find the first matching branch in program order. Finding the right branch means comparing the constructor of the matched value with the constructor of the branch. This requires checking for equality of the runtime unique identifiers, although in practice the *OCaml* compiler generates physical equality checks on the constructor blocks to save a few instructions.

1.4 Contributions

The reason why extensible variants are less efficient than other kind of variants when pattern-matching is because too little is known at compile time about the extensible constructors, so the generated code checks each constructor sequentially. This means that branch selection is done in linear time (on the number of branches). In this paper, we will focus on exploring solutions to reduce the branch selection cost, in particular by looking at how other variant types are compiled.

We present three compilation schemes that we think can increase performance of OCaml programs that heavily relies on extensible variants. A proof of concept for two of these schemes have been implemented in a fork[2] of the OCaml compiler and have been tested both on the compiler itself, its testsuite and dune. We also provide some benchmarks[3] that show improvement in performance with our compilation schemes.

2 Dynamic Dispatch

The main idea of this first compilation scheme is to build a decision tree mapping the variant's unique identifier to a branch. Since the identifiers are assigned during runtime we cannot build the tree statically. Instead, we will split the process in two parts:

- During compilation, we map every involved constructor to a static unique identifier, and update the pattern-matching by replacing the constructors with their corresponding identifiers as the discriminant. We then compile the map into an hard-coded associative list and generate a call to an initialization function. Finally, we generate a call to a runtime function that will fetch the branch identifier and the pattern-matching is compiled as an integer *switch* over the branch identifier.

- During runtime, we call an initialization function once (intuitively the first time this pattern-matching is computed). This function receives an associative list mapping constructor addresses to branch identifiers and will build a tree mapping constructor identifiers to branch identifiers.[4] When matching we fetch the identifier of the constructor and lookup in the tree to find the branch index, then jump to it.

[2]https://github.com/mcoulmance/ocaml/tree/extensible_variants
[3]https://github.com/mcoulmance/ocaml_benchmark
[4]We use the constructor's identifier rather than its address to avoid eventual issues with the garbage collector (i.e. during the compact phase).

```
let foo = function
  | A -> "A"
  | B -> "B"
  | C -> "C"
  | _ -> "unknown"
```

```
(let
   table =
     (apply CamlinternalExtension.init_table [(A, 3), (B, 2), (C, 1)])
   foo =
     (function param
       (switch (apply table param)
        case int 3: "A"
        case int 2: "B"
        case int 1: "C"
        default: "unknown")))
```

Figure 2. Pseudocode example for the dynamic dispatch scheme

2.1 Extension of the Standard Library

Since dispatch tables are initialized at runtime, we added a new module to the standard library to handle them. The tables are implemented as *RBtrees*[5] to ensure a logarithmic lookup time. To avoid exposing the table itself to the user, we hide it in the closure of a function that receives an associative list (binding extension constructors to integers), and returns a new function that takes the unique identifier of the constructor to lookup for and returns the identifier of its corresponding branch. The biggest advantage of this approach is that from the program perspective, we do not manipulate any complex data structure, we only call functions. This highly simplifies generated code.

2.2 Implementation

One important issue to address is dispatch table initialization. More precisely, *where* to initialize them. Indeed, initializing a table has a non-negligible runtime cost, and initializing a new table each time the same pattern-matching is computed would have a devastating impact on program performance. We therefore would like to move the initialization outside the functions so that it only needs to be executed once. We call this process *code lifting*. Care must however be taken not to lift the tables too high: the table initialization must occur when all the constructors are in scope. The implementation of this solution in the *OCaml* compiler is carried out in two passes:

A first top-down pass will mark all pattern-matching and generate the switch, first on an uninitialized abstract value. During this pass we also build an initialization environment, associating each constructor with the index of its corresponding branch. At the end of this first pass, we obtain an abstract syntax tree in which every pattern-matching of the relevant type has been replaced by an uninitialized switch.

A second bottom-up pass will back-propagate each initialization environment until we reach the definition of a constructor involved in one (or more) pattern-matching. At this point, we initialize every table that is related to this constructor, and we continue with the back-propagation. When initializing a table we first generate the list containing the

[5]The current implementation allows for experimenting with other data structures, eventually performing benchmarks to see which one fits best, but this goes out of the scope of this paper.

identifiers and the indices, then we generate a call to the runtime function and store its result in the variable used in the associated switch.

One of the advantages of this backpropagation method is that it allows us to simultaneously perform a second optimization: merging duplicate tables. Similar pattern-matching can generate identical dispatch table. In that case it is reasonable to get rid of the duplicates to save both computation time and memory. To do this we add a new step to the table initialization code to find duplicates. At the end of this step we obtain an associative list that binds initialization environments to a list of switch variables. All we need to do next is to generate a unique version of each table and bind it to all the switch variables.

2.3 Analysis

The advantage of this implementation is that it allows the code branch to be selected in $O(log(n))$ time instead of $O(n)$ in the current implementation. By using more efficient data structures, we could even get it done in $O(1)$. The main drawback is the memory consumption, which increases linearly with the pattern-matching size. Another drawback is that, although more efficient on paper, our implementation relies on a heap allocated data structure. Modern processors being particularly well optimized for branch prediction, it may be that in a pattern-matching with sufficiently few cases our implementation becomes less efficient than the original one (i.e. for pattern-matching with few branches, it may be cheaper to perform a linear search than to fetch memory from the heap). It is therefore necessary to decide on a threshold above which the tree traversal becomes more efficient, which is not trivial given that these types of parameters vary greatly depending on the architecture. As for now, the prototype adds an annotation to manually enable dynamic dispatch.

3 Hash Modulo Collisions

The main idea for this second solution is to make maximum use of the information known at compile time to reduce the work at runtime. The only reliable information available when compiling a pattern-matching is the name of the different constructors. This isn't much, but we can draw inspiration from the way polymorphic variants are compiled to try to optimize the branch-selection time.

3.1 Compilation of polymorphic variants

The main constraint when compiling polymorphic variants is that the constructors do not belong to any particular type. This means that the representation used for classic variants (where unique identifiers are assigned to each constructor of the type using a counting function) cannot be used here. To work around this problem, and to ensure that all occurrences of a polymorphic variant constructor will have the same identifier, the *OCaml* compiler uses a hash function on the constructor name. When compiling a pattern-matching over polymorphic variants, it builds a decision tree using the hash of matched constructors. The main advantage is that it allows generating reasonable code without adding too many constraints. Therefore, we wondered if it was possible to port this method to extensible variants.

Polymorphic variant constructors are represented by their hash value at runtime (with some tweaks for constructors with a payload that are not relevant here). This can cause issues when two constructor names have the same hash. To workaround this problem, the typechecker will ensure that every constructor in a given type have different hash values (a classic example of this issue is given in Fig. 3).

```
let foo = function `squigglier -> false | _ -> true ;;

(* Correct: "squigglier" and "jfla" have different hash values *)
foo `jfla ;;

(* Incorrect: "squigglier" and "premiums" have the same hash value,
   this will be rejected by the compiler *)
foo `premiums ;;
```

Figure 3. Classic example of hash collision with polymorphic variants

3.2 Porting to Extensible Variants

A first fundamental difference between polymorphic variants and extensible variants is the handling of hash collisions. Two constructors with identical hashes become indistinguishable from each other once compiled, which can potentially distort the match result. While polymorphic variants can make the reasonable assumption that users will be able to find new names to avoid collisions (and therefore reject them), we cannot. For instance a function can declare a first class module containing extensions, meaning that each call to the later will produce a new set of constructors with the exact same name. Rejecting such code would break backward compatibility (and reduce expressivity of extensible variants). Instead, after the hash selection we add a physical equality test to verify that the branch we selected corresponds to the constructor we are matching against. In a context where the pattern-matching includes several constructors known to have the same hash, we simply generate a test sequence on them. In all cases, we introduce an "else" branch that jumps to the default case, allowing us to maintain the correctness of the result in the event of a matching failure (corresponding to an unmatched constructor having the same hash as one of the branch selectors).

A second main difference with polymorphic variants is constructor rebinding. Indeed, generating the decision tree requires knowing the hashes of all the constructors present in the patterns. However, rebinded constructors cannot always be known when compiling a pattern-matching: if a rebinded constructor comes from a different module, it could be exposed in the signature as a normal constructor. In fact, at the moment the *OCaml* language does not even allow rebinding in signatures. This poses a major problem since we cannot know which constructors are rebindings and, by extension, which hashes to check for during pattern-matching. A first solution could be to extend the signature language to add the ability to express constructor rebinding. This would also require modifying the type checker to verify consistency between the signature and the implementation. However, this solution would break some pieces of existing code, which would be unfortunate. Instead, we decided to work on a special case of extensible types in which rebinding is simply prohibited. To enable this optimization, we must therefore annotate the type declaration (we currently use the attribute `[@strict]`). When a type is declared as strict, rebinding is prohibited (the type-checker has been updated to verify this constraint), which allows us to ensure that the hash values to use are known regardless of the compilation context. More precisely, accessing the hash of a constructor at runtime will always give the same value as computing the hash directly from the constructor's name.

Finally, we also have to deal with runtime representation. Since the *OCaml* runtime relies on the current representation we decided to simply add a new field to extensible constructor blocks, storing the hash of the (non-qualified) constructor name.

```
function
  | Eric_Cooper -> "Eric_Cooper"   (* Collision with NSaServ     *)
  | NSaServ -> "NSaServ"           (* Collision with Eric_Cooper *)
  | Jfla26 -> "Jfla26"             (* No collision here          *)
  | OCaml -> "OCaml"               (* No collision here          *)
  | _ -> "Unknown"
```

```
  (let foo = function param
     (if (!= (get_hash param) -332323982)
       (if (!= (get_hash param) 418689109)
         (if (!= (get_hash param) 691738988) (goto 2)
           (if (== param OCaml) "OCaml" (goto 2)))
         (if (== param Jfla26) "Jfla26" (goto 2)))
       (if (== param Eric_Cooper) "Eric_Copper"
         (if (== param NSaServ) "NSaServ" (goto 2))))
    with (2) "Unknown")
```

Figure 4. Pseudocode example for the hash modulo collisions scheme

3.3 Analysis

The main advantage of this implementation is that it allows branch-selection to be done in $O(log(n))$ time on average, like the dynamic dispatch implementation, but without increasing memory consumption. Furthermore, the code size does not increase significantly on average. The only case in which we could observe poorer performance would be if all the constructors being filtered have the same hash, in which case we would end up creating a cascade of conditionals as in the classic implementation. The main drawback of this implementation is the prohibition on rebinding. Anecdotal evidence suggest that rebinding is almost never used in practice, except for exceptions. We might even make `strict` mode the default in the future.

4 Results

Evaluating the performance of our two implementations wasn't particularly easy since extensible variants remains a little-used feature of the language. There is, to the best of our knowledge, no real project that uses big pattern-matching over extensible types in performance critical code.

We nevertheless developed some benchmarks to get an idea of the effectiveness of the *dynamic dispatch* and the *hash modulo collision* compilation schemes. The code to reproduce the benchmarks are available in a dedicated Git[6] repository. All the benchmarks presented here were performed on a machine equipped with an Intel Core Ultra 5 processor and 16 GB of RAM.

The first benchmark consists in replacing the OCaml compiler's abstract syntax trees with extensible types. The goal is to compare the different compilation speeds on the compiler module "typecore.ml". We chose this file in particular because it contains code complex enough to achieve interesting results. Of course, since we are replacing classic variants with extensible types, the performance of the generated compilers will be lower than that of the base compiler, but the goal here is to be able to compare the different implementations of

[6] https://github.com/mcoulmance/ocaml_benchmark

	Number of instructions	Difference	Time	difference
extensible types	$8.2 \cdot 10^9$	+0%	1.088s	+0%
dynamic dispatch	$9.1 \cdot 10^9$	+10.5%	1.358s	+24.8%
hash mod collisions	$7.9 \cdot 10^9$	−4.4%	1.084s	−0.4%

Figure 5. Benchmark of the compilation of the module "typecore.ml" using a patched version of the compiler

		Number of instructions	Difference	Time	Difference	
Bytecode	extensible types	$16 \cdot 10^9$	+0%	1.083s	+0%	Bytecode
	dynamic dispatch	$46 \cdot 10^9$	+188%	2.342s	+116%	
	hash mod collisions	$12.7 \cdot 10^9$	−20.5%	1.125s	+3.9%	
Native	extensible types	$1.8 \cdot 10^9$	+0%	0.264s	+0%	Native
	dynamic dispatch	$3.9 \cdot 10^9$	+113%	0.558s	+111.3%	
	hash mod collisions	$2 \cdot 10^9$	+11.4%	0.261s	−1.14%	

Figure 6. Benchmark of "deep" pattern-matching

extensible types with each other, not to compare them with classic variants (which would not make much sense). Figure 5 shows the result we obtained from these tests. We can see that performance are slightly better with the hash modulo collisions implementation but worse with dynamic dispatch. This can be explained by the fact that when traversing the table, we will have to load memory blocks located outside the cache many times (since the table nodes are allocated in the heap), but this operation is costly compared to simple arithmetic comparisons. We can conclude that, in a usage similar to that of classic variants (which is not the initial goal of extensible variants), it is useful but not necessary to optimize pattern-matching.

For the following two benchmarks, we tried to write a file with some of the most complex uses of extensible types that we could find. The code (available on the git) is constructed as follows: we use functors to dynamically extend an extensible type, on which we define a pattern-matching that recursively iterates through all the constructors already defined. The advantage of this example is that it allows us to check the behaviour of our implementations on some of the most problematic cases that we could end up having to compile.

We perform two benchmarks: In the first one (figure 6) we instantiate 10000 modules, launching at each new instantiation a pattern-matching on the constructor located lowest in the test sequence. In the second one (figure 7) we instantiate 10000 modules, launching at each new instantiation a pattern-matching on a constructor chosen randomly from among those so far defined.

These numbers show that using dynamic dispatch can incur a significant cost, but that

		Number of instructions	Difference	Time	Difference	
Bytecode	extensible types	$66.8 \cdot 10^9$	+0%	4.34s	+0%	Bytecode
	dynamic dispatch	$60.3 \cdot 10^9$	−9.9%	4.7s	+8.3%	
	hash mod collisions	$60.7 \cdot 10^9$	−9.1%	4.21s	−3%	
Native	extensible types	$4 \cdot 10^9$	+0%	0.908s	+0%	Native
	dynamic dispatch	$4.9 \cdot 10^9$	+18.4%	0.652s	−28.2%	
	hash mod collisions	$4.5 \cdot 10^9$	+8.97%	0.78s	−14.1%	

Figure 7. Benchmark of "randomized" pattern-matching

	Number of instructions	Difference	Time	difference
extensible types	$15 \cdot 10^9$	+0%	$2.07s$	+0%
dynamic dispatch	10^9	−90%	$0.1s$	−93%
hash mod collisions	$4.7 \cdot 10^8$	−97%	$0.04s$	−98%

Figure 8. Benchmark of the compilation of a huge pattern-matching

hash modulo collision is roughly similar to the default scheme.

Finally, we crafted an example for which we expect our implementations to perform significantly better than the default one. This example defines an extensible type, adds 1,000 constructors, and exports a function that pattern-matches on all those constructors, returning a different constant integer each time. This function is then called 10,000 times in a loop on a given element, and this is repeated 1,000 times with random elements. The results are shown in Fig. 8.

The benchmarks show that on usual code, even when forcing more extensible type use than normal, the dynamic dispatch scheme performs slightly but noticeably worse than the default one, but the hash modulo collision scheme is competitive and even a little better than the default one.

The pathological benchmark confirms the tendency, with dynamic dispatch introducing a massive overhead but hash modulo collision still comparable with the default scheme.

Finally, the best-case benchmark shows that we do achieve our goal of significant gains for large extensible types, by the expected margin for hash modulo collision (a 40 times speedup for 1,000 cases is consistent with $O(log(n))$ versus $O(n)$), and by some respectable if not as impressive margin for dynamic dispatch.

5 Conclusion and Future Work

We have presented several approaches to the compilation of pattern-matching on extensible types. Our approach focused on the admittedly uncommon case of pattern-matching on extensible types with many constructors, and we believe that we obtained satisfying results for these cases. Our hope is that future programmers could consider using large extensible types without fears of punitive overhead on pattern-matching functions.

The impact on existing programs, for the hash modulo collision approach, seems small enough to consider submitting it properly to the *OCaml* team.

Although this was not one of our initial objectives, we believe that the `[@strict]` attribute (and its associated checks) is valuable on its own, and plan to propose it for inclusion in the *OCaml* language independently.

A better way to handle extensible variants would be to split the set of constructors of a type in relation to the scope[7] in which they are defined. The idea would be to identify a constructor relatively to the scope in which it is defined, using a runtime-defined *scope identifier* and a compile-time defined *local* identifier. We could then generate a test sequence on the scope identifiers, then for each scope a symbol table using the local identifiers.

The main advantage of this solution is that it allows branch-selection to be done in linear time *on the number of scopes involved* which should be a relatively small number, or even in constant time if every involved constructor came from the same scope.

To implement this solution we would need to define a runtime notion of *scope*, to extend the typing environment with relevant information about extension constructors and to handle mismatches between structures and signatures (missing constructors, reordering). The results for our two schemes makes us confident that it is worth trying to implement this solution as well.

[7] Here we consider a scope corresponding to an *OCaml* module, but this definition may need to be refined later.

References

[FM01] Fabrice Le Fessant et Luc Maranget : Optimizing pattern matching. *In* Benjamin C. Pierce, éditeur : *Proceedings of the Sixth ACM SIGPLAN International Conference on Functional Programming (ICFP '01), Firenze (Florence), Italy, September 3-5, 2001*, pages 26–37. ACM, 2001.

[Ler90] Xavier Leroy : The ZINC experiment: an economical implementation of the ML language. Technical report 117, INRIA, 1990.

[Mar08] Luc Maranget : Compiling pattern matching to good decision trees. *In Proceedings of the 2008 ACM SIGPLAN Workshop on ML*, ML '08, page 35–46, New York, NY, USA, 2008. Association for Computing Machinery.